Aula 15 — Polimorfismo

> **A mesma mensagem, comportamentos diferentes**

Como Java decide qual implementação executar por uma referência de interface?

O papel é comum; as regras são diferentes

```
public interface Entrega {
    double calcularCusto(Pedido pedido);
}
```

Classe que cumpre Entrega	Custo neste recorte
`EntregaNormal`	`10.0`
`EntregaExpressa`	`25.0`

As regras são fixas: recebem o pedido, mas não consultam seus dados.

ATIVIDADE

SITUAÇÃO 1 — A MESMA VARIÁVEL

O que será impresso em cada chamada?

```
Pedido pedido = new Pedido();
Entrega entrega = new EntregaNormal();
System.out.println(entrega.calcularCusto(pedido));

entrega = new EntregaExpressa();
System.out.println(entrega.calcularCusto(pedido));
```

1. Quais valores aparecem, na ordem? Qual objeto recebe cada chamada?
2. O tipo declarado de `entrega` mudou?

SITUAÇÃO 1 — ACOMPANHE O OBJETO ALCANÇADO

10.0 e depois 25.0

Chamada	Objeto alcançado	Implementação
Primeira	`EntregaNormal`	devolve `10.0`
Segunda	`EntregaExpressa`	devolve `25.0`

O tipo declarado continua sendo **Entrega**.

A atribuição muda o objeto alcançado pela variável; cada objeto realiza sua regra.

ATIVIDADE

SITUAÇÃO 2 — UMA REFERÊNCIA FOI COPIADA

A segunda acompanha a primeira?

```
Pedido pedido = new Pedido();
Entrega primeira = new EntregaNormal();
Entrega segunda = primeira;
primeira = new EntregaExpressa();

System.out.println(primeira.calcularCusto(pedido));
System.out.println(segunda.calcularCusto(pedido));
```

1. Quais valores aparecem? Quantos objetos de entrega foram criados?
2. A última atribuição transforma a normal ou muda `segunda`?

SITUAÇÃO 2 — CADA VARIÁVEL GUARDA SUA REFERÊNCIA

25.0 e depois 10.0

- `primeira` alcança a expressa; `segunda` conserva a normal.
- Foram criados **dois objetos de entrega**.
- Copiar a referência não cria outro objeto.
- Reatribuir `primeira` não transforma a normal nem altera `segunda`.

Vamos representar o estado antes e depois da reatribuição.

Duas variáveis alcançam o mesmo objeto

```
Entrega primeira = new EntregaNormal();
Entrega segunda = primeira;
```

<svg
aria-
hidd
en="t
rue"
view
Box=
"0 0
72
100"

DEPOIS DE PRIMEIRA = NEW ENTREGAEXPRESSA()

Só o caminho de primeira mudou



O objeto normal continua sendo o mesmo alcançado por `segunda`.

ARMADILHA

A PREVISÃO DEPENDE DE SEPARAR VARIÁVEL E OBJETO

Reatribuir não transforma o objeto

```
primeira = new EntregaExpressa();
```

- Cria uma entrega expressa e guarda sua referência em `primeira`.
- Não transforma a entrega normal em expressa.
- Não faz outras variáveis acompanharem essa atribuição.

CONCEITO-CHAVE

DAR NOME AO COMPORTAMENTO OBSERVADO

Conceito-chave — polimorfismo

Podemos usar objetos de classes diferentes por um **mesmo tipo** e solicitar a **mesma operação**.

A implementação executada depende do objeto que recebe a chamada.

Aqui, referências `Entrega` permitem solicitar `calcularCusto(Pedido)` a objetos normais ou expressos.

COMO A CHAMADA CHEGA À IMPLEMENTAÇÃO

Essa escolha é o despacho dinâmico

```
entrega.calcularCusto(pedido)
```

Durante a execução, Java chega ao objeto referenciado por `entrega` e executa sua implementação do método de instância.

Polimorfismo não exige resultados diferentes: cada implementação pode produzir o resultado previsto por sua regra.

A expressa oferece uma operação própria

Considere este método público dentro de `EntregaExpressa`:

```
public void priorizarNaFila() {
    System.out.println("Entrega priorizada");
}
```

`priorizarNaFila()` não pertence ao contrato `Entrega`.

O objeto tem o método. Isso basta para chamá-lo por qualquer referência?

ATIVIDADE

SITUAÇÃO 3 — ANALISE AS CHAMADAS SEPARADAMENTE

O objeto tem; o tipo permite?

```
Pedido pedido = new Pedido();
Entrega entrega = new EntregaExpressa();

entrega.calcularCusto(pedido);
entrega.priorizarNaFila();
```

1. Quais chamadas compilam? Por quê?
2. Retirando a linha inválida, qual cálculo executa?
3. Com `EntregaExpressa expressa`, a prioridade poderia ser solicitada?

SITUAÇÃO 3 — DOIS CRITÉRIOS DIFERENTES

O contrato permite calcular, mas não priorizar

1. `calcularCusto(pedido)` compila; `priorizarNaFila()` não: o tipo declarado é `Entrega`, que só promete o cálculo.
2. Sem a linha inválida, executa `EntregaExpressa.calcularCusto`, que devolve **25.0**.
3. Sim. Uma referência declarada `EntregaExpressa` permite solicitar sua operação pública específica.

O objeto não perdeu o método; aquela referência não permite essa chamada.

JAVA EM FOCO

DUAS PERGUNTAS EM MOMENTOS DIFERENTES

Java em foco — compilação e execução

Na compilação

O tipo declarado da referência permite a chamada?

Na execução de uma chamada válida

Qual objeto recebe a chamada e qual implementação do método de instância executa?

O despacho dinâmico não torna válida uma chamada recusada pelo compilador.

O pedido guarda a entrega no fechamento

```
private Entrega entrega;
private boolean fechado;

public void fechar(Entrega entrega) {
    if (!fechado && entrega != null) {
        this.entrega = entrega;
        fechado = true;
    }
}
```

O primeiro fechamento válido conserva a escolha. A regra é a da Aula 14.

A consulta encaminha a solicitação

Dentro de `Pedido`:

```
public double calcularCustoEntrega() {
    return entrega.calcularCusto(this);
}
```

O campo `entrega` indica quem recebe o cálculo; `this` fornece o próprio pedido como argumento.

Neste estudo, essa consulta ocorre **depois de um fechamento válido**.

ATIVIDADE

SITUAÇÃO 4 — A VARIÁVEL MUDOU

A entrega do pedido também mudou?

```
Entrega entrega = new EntregaNormal();
Pedido pedido = new Pedido();
pedido.fechar(entrega);
entrega = new EntregaExpressa();

System.out.println(entrega.calcularCusto(pedido));
System.out.println(pedido.calcularCustoEntrega());
```

1. Quais valores aparecem? Quem recebe cada cálculo de custo?
2. Que atribuição mudaria o campo? Ela aconteceu após criar a expressa?

SITUAÇÃO 4 — CAMPO E VARIÁVEL SÃO LUGARES DISTINTOS

A expressa devolve 25.0; o pedido continua em 10.0

- A primeira consulta chega diretamente à **entrega expressa** da variável local.
- A segunda chega ao **pedido**, que solicita o custo à **entrega normal** guardada em seu campo.
- Mudar o campo exigiria uma atribuição dentro de `Pedido`, como `this.entrega = entrega`.
- Depois de criar a expressa, só a variável local foi reatribuída.

O campo continua apontando para a normal



O campo está contido no pedido; a variável local está fora dele.

ATIVIDADE

SITUAÇÃO 4 — CONTINUE COM O MESMO PEDIDO FECHADO

E se tentarmos fechar novamente?

A variável local `entrega` agora alcança a expressa.

```
pedido.fechar(entrega);
System.out.println(pedido.calcularCustoEntrega());
```

1. A chamada a `fechar` compila?
2. O campo muda para a expressa? Qual custo aparece e por quê?

O custo continua sendo 10.0

1. A chamada compila: `fechar` aceita `Entrega`.
2. O campo não muda: o pedido já está fechado, então `!fechado` é falso.

```
if (!fechado && entrega != null) {
    this.entrega = entrega;
    fechado = true;
}
```

A consulta continua chegando à entrega normal.

ARMADILHA

O CONTRATO NÃO CONTORNA REGRAS DO DOMÍNIO

Polimorfismo não troca o campo sozinho

- A variável local pode mudar sem alterar o campo de outro objeto.
- Aceitar um argumento não obriga o método a guardá-lo.
- O despacho usa o objeto alcançado pela referência da chamada.

`Pedido` continua controlando quando guarda a entrega.

Um teste protege variável versus campo

Em `PedidoTest`, com JUnit Jupiter e `assertEquals` configurados:

```
@Test
void reatribuirVariavelNaoTrocaEntregaDoPedido() {
    Entrega entrega = new EntregaNormal();
    Pedido pedido = new Pedido();
    pedido.fechar(entrega);
    entrega = new EntregaExpressa();
    assertEquals(25.0, entrega.calcularCusto(pedido));
    assertEquals(10.0, pedido.calcularCustoEntrega());
}
```

E se o pedido escolhesse a fórmula?

Proposta em linguagem natural, dentro de `Pedido`:

```
se for normal: devolver 10
senão, se for expressa: devolver 25
senão, se for retirada local: devolver 0
```

Na colaboração atual, `Pedido` solicita:

```
return entrega.calcularCusto(this);
```

ATIVIDADE

UMA NOVA ALTERNATIVA TESTA A DISTRIBUIÇÃO DAS REGRAS

Chegou a entrega agendada: custo 15.0

1. Na proposta com condicionais, onde acrescentar a nova regra?
2. Na colaboração pelo contrato, quem implementa o cálculo? `Pedido` precisa mudar?
3. Quem cria e fornece a entrega, se o pedido não identifica sua classe?

A NOVA REGRA FICA NA NOVA IMPLEMENTAÇÃO

O pedido continua solicitando a mesma operação

```
public class EntregaAgendada implements Entrega {
    public double calcularCusto(Pedido pedido) {
        return 15.0;
    }
}
```

1. Com condicionais, seria preciso acrescentar um caso em `Pedido`.
2. Pelo contrato, a nova classe calcula; `Pedido` permanece igual.
3. O código que monta o cenário, como `Main`, cria e fornece o objeto.

Uma sequência pode reunir objetos diferentes

Vamos comparar custos diretamente, sem fechar o pedido com várias entregas.

No `main`:

```
Pedido pedido = new Pedido();
List<Entrega> alternativas = new ArrayList<>();
alternativas.add(new EntregaNormal());
alternativas.add(new EntregaExpressa());
```

O tipo dos elementos agora é o contrato `Entrega`.

Java em foco — lista e percurso

```
import java.util.ArrayList;
import java.util.List;
```

- `List<Entrega>` declara a lista de referências pelo contrato.
- `new ArrayList<>()` cria uma lista inicialmente vazia.
- `add(...)` inclui a referência no fim da sequência.
- `for (Entrega entrega : alternativas)` percorre os elementos na ordem.

Não precisamos de uma lista diferente para cada classe de entrega.

ATIVIDADE

A LISTA CONTÉM NORMAL E EXPRESSA, NESSA ORDEM

Qual regra executa a cada passagem?

```
for (Entrega entrega : alternativas) {
    System.out.println(entrega.calcularCusto(pedido));
}
```

1. Qual é a saída? O tipo declarado de `entrega` muda?
2. Precisamos identificar a classe do elemento dentro do laço?
3. E com lista vazia ou com a mesma referência em duas posições?

O PERCURSO REPETE A CHAMADA; O OBJETO PODE VARIAR

Uma chamada por posição percorrida

1. Saída: **10.0 e depois 25.0**. A variável permanece declarada `Entrega`.
2. Não. A chamada pelo contrato executa a implementação do objeto atual.
3. Lista vazia não executa o corpo; a mesma referência em duas posições recebe duas chamadas.

O número de chamadas depende das posições percorridas, não da quantidade de objetos distintos.

E se a mesma chamada alterasse outro objeto?

Um pequeno jogo:

- O personagem começa com **50 pontos de vida**.
- A poção solicita recuperar **20 pontos**.
- A armadilha solicita perder **10 pontos**.

O personagem controla sua vida; cada efeito decide o que solicitar a ele.

A CAPACIDADE COMUM É APLICAR UM EFEITO

O contrato recebe o personagem afetado

Em `Efeito.java`:

```
public interface Efeito {
    void aplicarEm(Personagem personagem);
}
```

O método não devolve um custo: pode produzir uma mudança de estado por colaboração.

Vida inicial e operações públicas

Recorte de `Personagem.java`:

```
private int vida;
public Personagem() { vida = 50; }
public int getVida() { return vida; }

public void recuperarVida(int pontos) {
    vida += pontos;
}
public void perderVida(int pontos) {
    vida -= pontos;
}
```

O PRÓPRIO PERSONAGEM OFERECE O ALVO AO EFEITO

Receber encaminha a solicitação

Dentro de `Personagem`:

```
public void receber(Efeito efeito) {
    efeito.aplicarEm(this);
}
```

`this` é o personagem que recebeu a chamada.

O método usa o efeito recebido naquela chamada e não o guarda em um campo.

CADA EFEITO REALIZA SUA COLABORAÇÃO

A poção solicita recuperação

Em `Pocao.java`:

```
public class Pocao implements Efeito {
    public void aplicarEm(Personagem personagem) {
        personagem.recuperarVida(20);
    }
}
```

O efeito não acessa o campo privado `vida`: solicita uma operação ao personagem.

A armadilha cumpre o mesmo contrato

Em `Armadilha.java`:

```
public class Armadilha implements Efeito {
    public void aplicarEm(Personagem personagem) {
        personagem.perderVida(10);
    }
}
```

As classes de efeito escolhem a solicitação; o personagem altera sua própria vida.

ATIVIDADE

O MESMO PERSONAGEM RECEBE OS DOIS EFEITOS

Quais estados serão observados?

```
Personagem personagem = new Personagem();
Efeito efeito = new Pocao();
personagem.receber(efeito);
System.out.println(personagem.getVida());

efeito = new Armadilha();
personagem.receber(efeito);
System.out.println(personagem.getVida());
```

Quais valores aparecem? Por que o mesmo `receber(...)` produz outro efeito? Qual objeto muda de estado?

A vida passa por 70 e depois 60

Chamada	Efeito alcançado	Solicitação ao personagem	Vida
Primeira	`Pocao`	recuperar `20`	`70`
Segunda	`Armadilha`	perder `10`	`60`

O parâmetro de `receber(...)` agora alcança outro efeito; o despacho chega à outra implementação de `aplicarEm(...)`.

O estado alterado é a vida do **mesmo personagem**.

ATIVIDADE

ALÉM DE PREVER O VALOR, LOCALIZE OS RECEPTORES

Quem recebe cada chamada da poção?

```
personagem.receber(efeito);  // efeito é a poção
```

Dentro de `receber`:

```
efeito.aplicarEm(this);
```

Quem recebe `receber`, `aplicarEm` e `recuperarVida(20)`?
O que `this` representa nessa passagem?

O caminho da poção até a vida

```
Main solicita receber(poção) ao personagem
Personagem solicita aplicarEm(personagem) à poção
Pocao solicita recuperarVida(20) ao personagem
Personagem altera sua vida de 50 para 70
```

`this` , dentro de `Personagem.receber` , é esse personagem, fornecido como argumento à poção.

ATIVIDADE

CONTINUE COM A VIDA EM 70 E EFEITO SENDO A ARMADILHA

Reconstrua o segundo caminho

```
personagem.receber(efeito);
```

Desenhe ou escreva a sequência de chamadas até a alteração da vida.

Inclua os receptores, a operação solicitada e a vida antes e depois.

O caminho da armadilha até a vida

```
Main solicita receber(armadilha) ao personagem
Personagem solicita aplicarEm(personagem) à armadilha
Armadilha solicita perderVida(10) ao personagem
Personagem altera sua vida de 70 para 60
```

O personagem usa o novo argumento. `receber(...)` não guardou a poção anterior.

COMPARE COM A ENTREGA GUARDADA EM PEDIDO

O contrato não decide quando guardar o colaborador

Modelo	Quando recebe?	Guarda para depois?
`Pedido`	primeiro fechamento válido	sim, no campo `entrega`
`Personagem`	cada chamada a `receber`	não, usa o argumento

As regras dos métodos explicam essa diferença.

O polimorfismo permite a chamada pelos contratos nos dois casos.

ATIVIDADE

DIAGNOSTICAR UMA PROPOSTA PARA O JOGO

Precisamos descobrir a classe do efeito?

Uma sugestão afirma:

> “Dentro de receber, precisamos descobrir se o efeito é poção ou armadilha para escolher o que fazer.”

Essa identificação é necessária? Como a versão atual chega ao comportamento correto? Quem controla a vida?

O personagem pode solicitar aplicarEm(this)

- Não é necessário identificar a classe dentro de `receber`.
- A implementação do objeto recebido solicita recuperação ou perda.
- O personagem continua controlando `vida` por suas operações públicas.

```
public void receber(Efeito efeito) {
    efeito.aplicarEm(this);
}
```

ATIVIDADE

EVOLUIR A SOLUÇÃO SEM RECEBER O NOVO CÓDIGO PRONTO

Uma fonte de cura recupera 5 pontos

O mesmo personagem terminou com **60 pontos**.

1. Que contrato `FonteDeCura` deve cumprir e qual operação implementar?
2. O que deve solicitar ao personagem?
3. `Personagem.receber` ou `Efeito` precisam mudar?
4. Onde o novo objeto é criado e qual vida será observada?

A NOVA IMPLEMENTAÇÃO INCORPORA A VARIAÇÃO

FonteDeCura cumpre Efeito; a vida chega a 65

```
public class FonteDeCura implements Efeito {
    public void aplicarEm(Personagem personagem) {
        personagem.recuperarVida(5);
    }
}
```

- Implementa `aplicarEm(Personagem)` e solicita `recuperarVida(5)`.
- `Personagem.receber` e `Efeito` permanecem iguais.
- `Main` cria e fornece a fonte: `personagem.receber(new FonteDeCura())`.
- O mesmo personagem passa de **60 para 65**.

SÍNTESE

DA CHAMADA AO RESULTADO OU À MUDANÇA DE ESTADO

Quatro perguntas para analisar a execução

1. O tipo declarado permite a chamada?
2. Qual objeto a referência alcança?
3. Qual implementação do método de instância executa?
4. Qual resultado ou mudança de estado ela produz?

Reatribuir uma variável não transforma objetos nem troca outros campos.

A mesma chamada, vários canais

Evoluir o sistema de lembretes com:

```
public void avisarPorTodos(List<Notificador> notificadores)
```

- Um envio por posição, na ordem da lista.
- Contar cada envio e preservar o canal inicial.
- Lista vazia não envia; referência repetida recebe outro envio.
- Ampliar os testes e preservar a lista recebida.